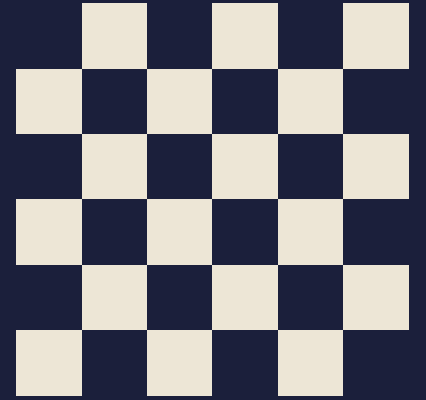


FUNDAMENTOS DE LA PROGRAMACIÓN

Bloque 2

Control de flujo



Objetivos del bloque



Tomar decisiones en el código

Aplicar estructuras condicionales (if / elif / else) para que un programa se comporte de forma distinta según los datos.



Repetir tareas sin duplicar código

Utilizar bucles (for, while) para ejecutar una acción varias veces, sustituyendo la repetición manual del Bloque 1.

Contenidos

1

2.1 Condicionales: if, elif, else

2

2.2 Indentación: los bloques de código en Python

3

2.3 Bucles for y la función range()

4

2.4 Bucles while

5

2.5 break y continue

6

2.6 Bucles anidados

7

2.7 Errores comunes y buenas prácticas

Cierre · Refactorizamos el tablero del Bloque 1 con bucles

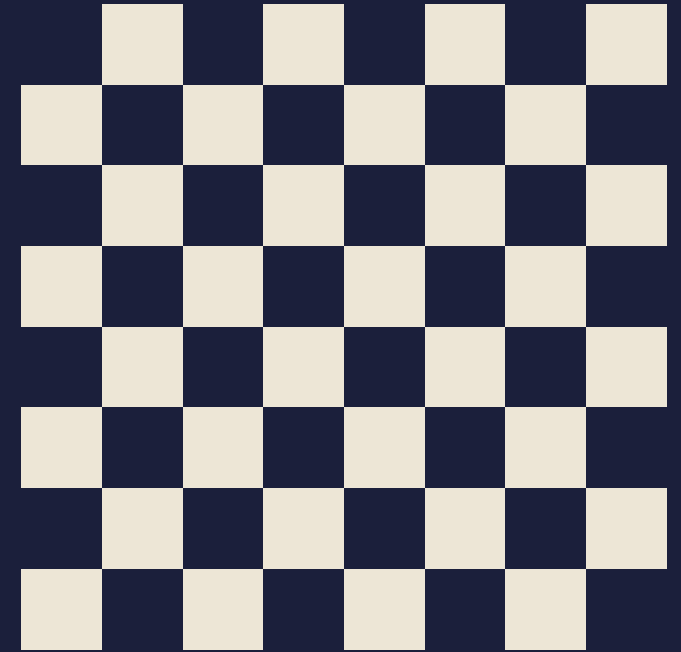
¿RECUERDAS ESTO?

6 líneas para dibujar una fila de 3 casillas

```
print(".", end="")  
print("#", end="")  
print(".", end="")  
...
```

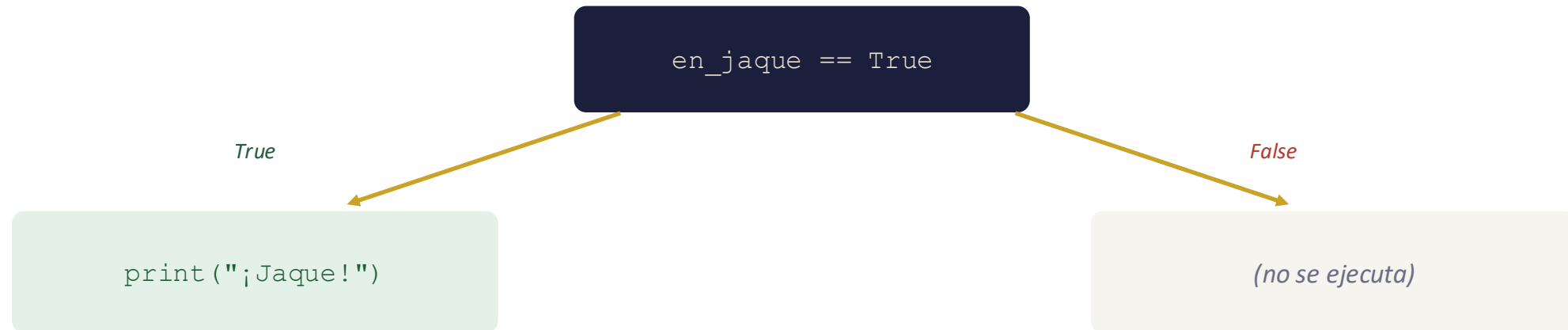


En este bloque aprendemos a decirle a Python "repite esto N veces" en una sola línea.



¿Qué es una condicional?

Una condicional hace que un bloque de código se ejecute solo si se cumple una condición (una expresión que da True o False). Es la primera vez que el programa "decide" en vez de ejecutar todo, siempre, de arriba abajo.



El resto del programa sigue igual, se ejecute o no ese bloque: la condicional decide solo lo que hay dentro de ella.

La sentencia if

if condición:

```
fila: int = 4
columna: int = 9

if columna > 7:
    print("La columna está fuera del tablero")
```

Los dos puntos y la sangría son parte de la sintaxis

Los dos puntos (:) al final de la línea del if son obligatorios.

Todo lo que pertenece al bloque "si se cumple" va sangrado (indentado) debajo — normalmente 4 espacios. En Python la sangría no es una cuestión de estilo: **es la sintaxis que delimita el bloque** (lo veremos en detalle en el siguiente apartado).

elif y else: más de dos caminos

```
resultado: str = ""
puntuacion: int = 7

if puntuacion >= 9:
    resultado: str = "Sobresaliente"
elif puntuacion >= 7:
    resultado: str = "Notable"
elif puntuacion >= 5:
    resultado: str = "Aprobado"
else:
    resultado: str = "Suspenso"
```

Python evalúa las condiciones en orden y ejecuta solo la primera que sea True (el resto se ignora, aunque también se cumplieran). **else** no lleva condición: es "en cualquier otro caso".

Condiciones compuestas en un if

Los operadores lógicos del Bloque 1 (and, or, not) se combinan con if para condiciones más ricas:

```
fila: int = 7
columna: int = 3

if (fila >= 0) and (fila <= 7) and (columna >= 0) and (columna <= 7):
    print("Posición dentro del tablero")
else:
    print("Posición fuera del tablero")
```

¿Te suena esta condición? Es exactamente dentro_del_tablero del Bloque 1 — ahora, en vez de solo calcularla, la usamos para decidir qué hacer.

match / case: el "switch" de Python

Cuando comparas UNA misma variable con muchos valores posibles, `match` puede sustituir a una cadena larga de `elif`:

```
pieza: str = "alfil"
valor: int = 0

match pieza:
    case "peon":
        valor: int = 1
    case "caballo" | "alfil": # varios valores, un caso
        valor: int = 3
    case _:                  # "en cualquier otro caso", como else
        valor: int = 0
```

⚠ Solo disponible desde Python 3.10

match/case se incorporó en octubre de 2021: en Python 3.6, 3.8 o 3.9 no existe y da `SyntaxError`. Comprueba tu versión con `python --version` antes de usarlo. Si tienes duda o trabajas en un entorno con una versión antigua, usa `if/elif/else`: hace exactamente lo mismo y funciona en cualquier versión.

Condicionales anidados: if dentro de if

No es lo mismo que "varias condiciones con and": aquí un if COMPLETO vive dentro del bloque de otro if.

```
fila: int = 3
columna: int = 9

if columna >= 0 and columna <= 7:
    if fila >= 0 and fila <= 7:
        print("Posición válida")
    else:
        print("Fila fuera de rango")
else:
    print("Columna fuera de rango")
```

Con columna = 9, ni siquiera se llega a mirar la fila: el if de fila está DENTRO del bloque "columna válida". Si la columna ya falla, Python no entra ahí — por eso el mensaje es "Columna fuera de rango", no "Fila fuera de rango", aunque la fila también pudiera estar mal.

Indentación: los bloques de código en Python

Muchos lenguajes marcan un bloque con llaves { }. Python usa la sangría (espacios al principio de línea) — no es estético, es sintaxis obligatoria.

CORRECTO

```
if column > 7:
    print("Fuera")
    print("del tablero")
```

INCORRECTO (IndentationError)

```
if column > 7:
print("Fuera")
    print("del tablero")
```

Reglas prácticas

Usa siempre 4 espacios por nivel de sangría (VS Code lo hace solo al pulsar Tab).

No mezcles espacios y tabulaciones en el mismo archivo: Python los distingue y puede dar error aunque "se vea" igual.

Todas las líneas de un mismo bloque deben tener exactamente la misma sangría.

Ahora te toca a ti

Clasifica la posición (*≈ 6 minutos*)

Declara `fila: int` y `columna: int` con los valores que quieras. Usando `if/elif/else`, imprime "Esquina" si `fila` y `columna` son 0 o 7 a la vez, "Borde" si solo una de las dos lo es, y "Centro" en cualquier otro caso.

```
fila: int = 0
columna: int = 7
# completa con if / elif / else...
```

Pista

Una esquina cumple `(fila == 0 or fila == 7) and (columna == 0 or columna == 7)`. Empieza comprobando esa condición primero, en el `if`.

Cambia los valores de `fila` y `columna` varias veces y comprueba que las tres frases salen cuando toca.

¿Qué es un bucle?

Un **bucle** repite un bloque de código varias veces, sin tener que escribirlo una y otra vez. Es la solución al problema que arrastramos desde el Bloque 1.

ANTES (Bloque 1)

```
print(".", end="")  
print("#", end="")  
print(".", end="")  
print("#", end="")  
print(".", end="")  
print("#", end="")
```

AHORA (Bloque 2)

```
casillas: str = "." * 3  
print(casillas, end="")  
  
# o, casilla a casilla:  
for i in range(3):  
    print(".", end="")
```

*Ya conocías * para repetir cadenas (Bloque 1). El bucle for hace lo mismo pero con cualquier bloque de código, no solo con texto.*

for y range()

for *variable* **in** *range(n)*:

```
for i in range(3):  
    print(i)  
# 0  
# 1  
# 2
```

range(n) va de 0 a n-1, nunca llega a n

range(3) genera 0, 1, 2 — tres valores, no incluye el 3. Es el mismo "empezar en 0" del Bloque 1 (índices de cadenas, columnas del tablero).

range(2, 8) → 2, 3, 4, 5, 6, 7 (inicio incluido, fin excluido).

range(0, 8, 2) → 0, 2, 4, 6 (con paso de 2).

Excepción a "anota siempre": la variable del for (*i*, *fila*, *columna*...) nunca lleva anotación — *for i: int in range(3):* es un error de sintaxis. Python la infiere sola.

Recorrer una cadena con for

Un for también puede recorrer directamente los caracteres de una cadena, sin pasar por range():

```
casilla: str = "e4"  
  
for caracter in casilla:  
    print(caracter) # 'e' luego '4'
```

¿Cuándo usar cada forma?

for i in range(n): cuando necesitas el índice (por ejemplo, para calcular una posición o comparar con otra fila).

for elemento in cadena: cuando solo te interesa el valor de cada carácter, no su posición.

Ahora te toca a ti

Cuenta atrás de piezas (*≈ 5 minutos*)

Usando un solo for con range(), imprime los números del 8 al 1 (uno por línea) — como si fueran las filas del tablero de arriba abajo.

```
for fila in range(???):  
    print(???)
```

Pista

range() no admite ir hacia atrás con solo dos argumentos — necesitas un tercero, el **paso**, en negativo: `range(8, 0, -1)`.

Reto extra: consigue el mismo resultado con range(1, 9) e imprimiendo 9 - fila.

Bucles while

while *condición*: repite el bloque MIENTRAS la condición sea True.

```
turno: int = 1

while turno <= 3:
    print(f"Turno {turno}")
    turno: int = turno + 1  # imprescindible: si no, el bucle no acaba nunca
```

for vs while: ¿cuál uso?

for cuando sabes de antemano cuántas veces se repite (8 filas, cada carácter de una palabra...).

while cuando no lo sabes de antemano: depende de una condición que cambia durante la ejecución ("hasta que el usuario escriba 'salir'", "mientras la vida sea mayor que 0").

La trampa: el bucle infinito

Antes de ejecutar: ¿qué va a pasar aquí?

```
turno: int = 1

while turno <= 3:
    print(f"Turno {turno}")
```

Nunca se actualiza "turno": el bucle no termina jamás

Es el error más habitual con while: olvidar la línea que acerca la condición a False. El programa se queda "colgado" imprimiendo Turno 1 para siempre.

Si te pasa: `Ctrl + C` en la terminal detiene la ejecución a la fuerza.

Regla práctica: todo while necesita, dentro de su bloque, algo que eventualmente haga falsa la condición.

break: salir del bucle antes de tiempo

break corta el bucle inmediatamente, aunque la condición siguiera siendo True (o quedaran vueltas de range()).

```
for columna in range(8):  
    if columna == 4:  
        print("Encontrada en la columna 4")  
        break # no sigue hasta la 7  
print(columna) # 0, 1, 2, 3 (y ya no más)
```

Uso típico: dejar de buscar en cuanto encuentras lo que necesitabas, sin recorrer el resto por nada.

continue: saltar a la siguiente vuelta

continue no corta el bucle: salta el resto del código de esa vuelta y pasa directamente a la siguiente.

```
for columna in range(8):  
    if columna % 2 == 0:  
        continue # se salta las columnas pares  
    print(columna) # 1, 3, 5, 7
```

break vs continue

break = "termina el bucle ya" · **continue** = "esta vuelta no, pero sigue con las demás".

Bucles anidados: un for dentro de otro

Para recorrer algo con dos dimensiones (filas Y columnas), se anida un for dentro de otro: por cada vuelta del externo, el interno se ejecuta completo.

```
for fila in range(3):          # bucle externo
    for columna in range(3):    # bucle interno
        print(f"({fila},{columna})", end=" ")
    print() # salta de línea al terminar cada fila
```

Salida: (0,0) (0,1) (0,2) ↵ (1,0) (1,1) (1,2) ↵ (2,0) (2,1) (2,2)

Por cada una de las 3 vueltas del externo, el interno se ejecuta sus 3 vueltas completas: $3 \times 3 = 9$ impresiones en total.

Bucles anidados: todo el tablero de una vez

```
casilla_clara: str = "."
casilla_oscura: str = "#"

for fila in range(8):
    for columna in range(8):
        if (fila + columna) % 2 == 0:
            print(casilla_clara, end="")
        else:
            print(casilla_oscura, end="")
    print() # salto de línea al final de cada fila
```

8 líneas de código sustituyen las $8 \times 8 = 64$ llamadas a `print()` que habría hecho falta escribir a mano en el Bloque 1. *Esto es exactamente lo que decíamos que ganaríamos con los bucles.*

Bucles y condicionales anidados, juntos

El ejemplo más completo del bloque: dos bucles anidados recorren todo el tablero, y dentro, condicionales anidados deciden qué es cada casilla.

```
for fila in range(8):
    for columna in range(8):
        if fila == 0 or fila == 7:
            if columna == 0 or columna == 7:
                print("Esquina", end=" ")
            else:
                print("Borde ", end=" ")
        else:
            if columna == 0 or columna == 7:
                print("Borde ", end=" ")
            else:
                print("Centro ", end=" ")
    print()
```

Cuenta los niveles de sangría de la línea más profunda (son 4) antes de ejecutar: es fácil perderse aquí.

Ahora te toca a ti

Triángulo de asteriscos (*≈ 7 minutos*)

Con dos bucles for anidados (sin usar el operador `*`), imprime un triángulo de 5 filas: la fila 1 tiene un asterisco, la fila 2 tiene dos, y así hasta la fila 5.

```
*  
* *  
* * *  
* * * *  
* * * * *
```

Pista

El bucle externo recorre las filas (1 a 5). El interno imprime tantos asteriscos como indique esa fila — `range(fila)` te da justo esa cantidad de vueltas.

Reto extra: consíguelo también usando el operador `*` de cadenas, en una sola línea por fila (sin bucle interno).

Errores comunes y buenas prácticas



Bucle infinito

En un while, olvidar la línea que acerca la condición a False.



Off-by-one

Olvidar que range(n) llega hasta n-1, no hasta n (falta o sobra una vuelta).



Sangría inconsistente

Mezclar espacios y tabulaciones, o desalinear líneas del mismo bloque.



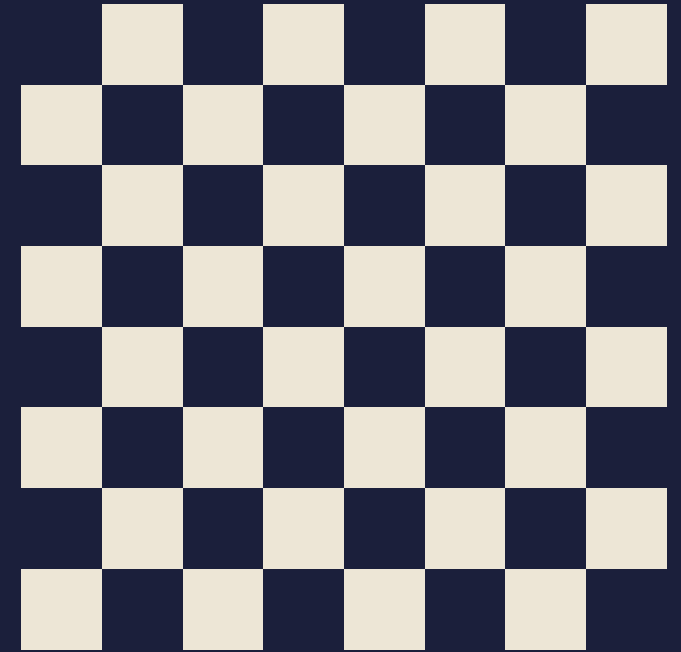
break mal ubicado

En bucles anidados, break solo sale del bucle más interno, no de todos.

CIERRE DEL BLOQUE

Refactorizamos el tablero

Volvemos al programa del Bloque 1 y lo reescribimos con todo lo aprendido aquí:
mismo resultado, muchísimo menos código.



Antes: el tablero del Bloque 1 (fragmento)

```
fila_empieza_clara: str = (clara_ancha + oscura_ancha) * 4
fila_empieza_oscura: str = (oscura_ancha + clara_ancha) * 4

print(" ", LETRAS_COLUMNAS)
print("8", fila_empieza_clara)
print("7", fila_empieza_oscura)
print("6", fila_empieza_clara)
print("5", fila_empieza_oscura)
print("4", fila_empieza_clara)
print("3", fila_empieza_oscura)
print("2", fila_empieza_clara)
print("1", fila_empieza_oscura)
# 8 líneas casi idénticas, una por fila...
```

Después: con bucles

```
print(" ", LETRAS_COLUMNAS)

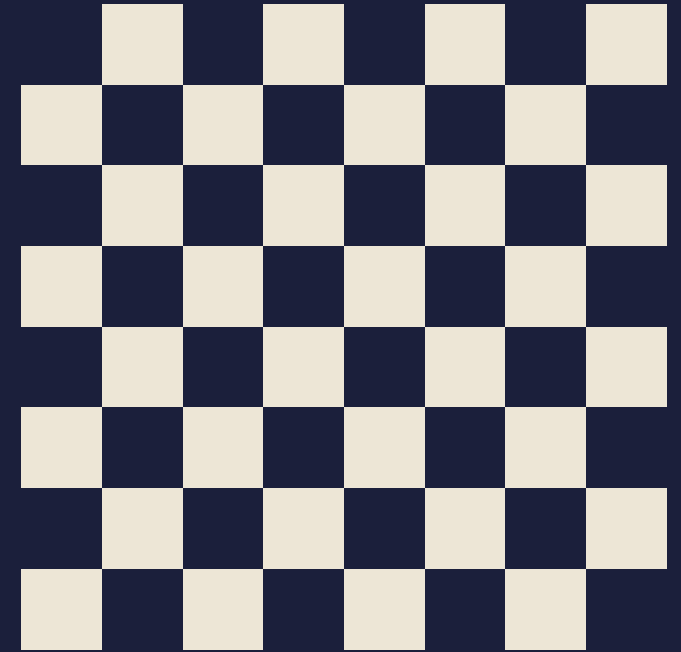
for numero_fila in range(8, 0, -1):
    if numero_fila % 2 == 0:
        print(numero_fila, fila_empieza_clara)
    else:
        print(numero_fila, fila_empieza_oscura)
```

Mismo resultado exacto, con un for y un if en vez de 8 líneas repetidas casi idénticas. **Esto es lo que ganamos al aprender control de flujo.**

MIRANDO AL FUTURO DEL PROYECTO

La máquina de estados

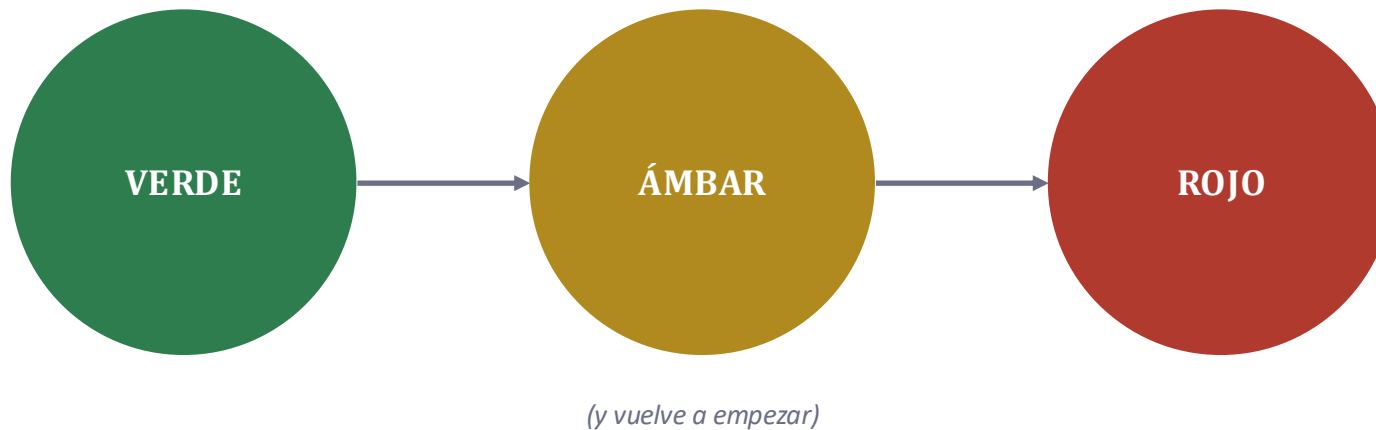
El bucle principal de todo el proyecto de ajedrez se va a construir con exactamente lo que acabas de aprender: un while y varios if/elif.



¿Qué es una máquina de estados?

Una máquina de estados es un sistema que, en cada momento, está en UNO de varios "estados" posibles — y solo puede pasar de uno a otro cuando ocurre algo concreto.

Un ejemplo que ya conoces: un semáforo



En VERDE, un semáforo solo puede pasar a ÁMBAR — nunca directamente a ROJO. *Cada estado define qué transiciones son posibles: eso es lo que lo hace una "máquina" y no un simple contador.*

Los estados de una partida de ajedrez

Nuestro proyecto tendrá sus propios estados. Una primera aproximación, muy simplificada:

turno_blancas

Esperando que las blancas elijan su movimiento

turno_negras

Esperando que las negras elijan su movimiento

jaque

El rey en juego está amenazado

fin_partida

Jaque mate, tablas, o abandono

Cada entrega irá añadiendo estados nuevos y afinando las condiciones que hacen pasar de uno a otro — pero el esqueleto (un while con if/elif) no cambiará.

El bucle principal del proyecto

```
estado: str = "turno_blancas"

while estado != "fin_partida":

    if estado == "turno_blancas":
        # pedir y aplicar el movimiento de las blancas
        estado: str = "turno_negras"

    elif estado == "turno_negras":
        # pedir y aplicar el movimiento de las negras
        estado: str = "turno_blancas"

    elif estado == "jaque":
        # avisar y volver al turno correspondiente

    ...
```

Este while no se parece al de "cuenta hasta 5": no cuenta nada, solo sigue dando vueltas mientras la partida no haya terminado. Cada vuelta completa mira en qué estado está el juego (if/elif) y decide qué hacer — este es, literalmente, el esqueleto que vais a ir rellenando entrega a entrega.

Unicode: más allá del teclado

Unicode es el estándar que usa Python para representar CUALQUIER carácter: alfabetos de todos los idiomas, símbolos matemáticos y, entre otras muchas cosas, las 12 piezas de ajedrez. Cada carácter tiene un número único, su **punto de código** (p. ej. U+2654), escrito en hexadecimal.

Dos formas equivalentes de escribir el mismo carácter en Python:

```
# Opción A: escribir el símbolo directamente
rey_blanco: str = "♔"

# Opción B: con su punto de código, en hexadecimal
rey_blanco: str = "\u2654" # el mismo carácter ♔
```

La opción B es más fiable: `\u2654` se escribe con las teclas normales de cualquier teclado, sin depender de copiar y pegar. Python lo convierte automáticamente en ♔ al ejecutar el programa — el resultado es exactamente el mismo.

Las piezas de ajedrez en Unicode

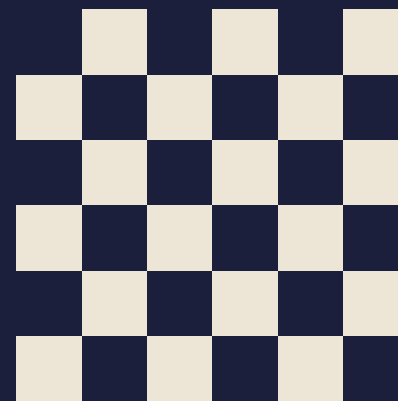
Pieza	Blanca	Código (blanca)	Negra	Código (negra)
Rey		<code>\u2654</code>		<code>\u265A</code>
Dama		<code>\u2655</code>		<code>\u265B</code>
Torre		<code>\u2656</code>		<code>\u265C</code>
Alfil		<code>\u2657</code>		<code>\u265D</code>
Caballo		<code>\u2658</code>		<code>\u265E</code>
Peón		<code>\u2659</code>		<code>\u265F</code>

Comprueba antes que nada que estos 12 símbolos se ven bien en tu propia terminal (la integrada de VS Code los muestra correctamente) — los necesitarás en la Entrega 1.

Resumen del Bloque 2

- ✓ Escribir condicionales con if, elif y else
- ✓ Anidar condicionales y bucles cuando hace falta
- ✓ Recorrer un rango de números o una cadena con for
- ✓ Escribir bucles while y evitar bucles infinitos
- ✓ Anidar bucles for para recorrer dos dimensiones (filas y columnas)
- ✓ Entender una máquina de estados como un while + if/elif
- ✓ Combinar condiciones con and, or y not dentro de un if
- ✓ Respetar la indentación como sintaxis, no como estilo
- ✓ Usar range() con inicio, fin y paso
- ✓ Usar break y continue con criterio
- ✓ Reconocer los errores más comunes de control de flujo
- ✓ Escribir caracteres Unicode en Python (directos o con \u)

Con esto ya tienes todo lo necesario para dejar de repetir código a mano — el resto del proyecto del ajedrez se apoyará constantemente en bucles y condicionales.



Próximo bloque

Bloque 3 · Estructuras de datos

Listas, tuplas y diccionarios: aquí arranca la primera entrega real del proyecto de ajedrez.